CHAMELEON: Hierarchical Clustering with Dynamic Modeling

Graph-Based Clustering: General Concepts

- Graph-Based clustering needs a proximity matrix
  - Each edge between two nodes has a weight which is the proximity between the two points
- Sparsify the proximity graph
- Define a similarity measure between two objects
  - Based on the number of nearest neighbors that they share
  - Useful for overcoming problems with high dimensionality and clusters of varying density
- Use proximity graph to provide evaluation of whether two clusters should be merged
  - two clusters are merged only if the resulting cluster will have characteristics similar to the original two clusters

Sparsification

- The *m* by *m* proximity matrix for *m* data points can be represented as a dense graph and the weight of the edge between any pair of nodes reflects their pairwise proximity
- Every object has some level of similarity to every other object
  - This property can be used to sparsify the proximity graph (matrix)
  - By setting low-similarity values to 0 before beginning the actual clustering process
- Many methods for sparsification may be performed:
  - By breaking all links that have a similarity below a specified thereshold
  - By keeping only links to the k-nearest neighbors of point (k-NNG)

k-nearest neighbors graph (k-NNG)

p and q are connected if q is among the top k closest neighbors of p

Sparsification: Several beneficial effects

- **Data size is reduced**
  - The amount of time required to cluster the data is drastically reduced
  - Sparsification can eliminate more than 99% of the entries in a proximity matrix
- **Clustering often works better**
  - Sparsification techniques keep the connections to their nearest neighbors of an object while breaking the connections to more distant objects
  - This reduces the impact of noise and outliers and sharpens the distinction between clusters
- **Graph partitioning algorithms can be used**
  - CHAMELON and OPOSSUM Clustering

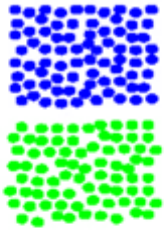
Graph-Based Clustering: CHAMELEON

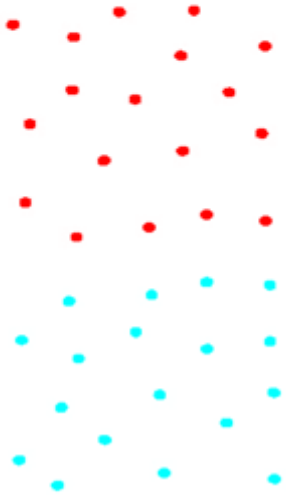
- Introduced in 1999 by Han and Kyarpis
- An Agglomerative Hierarchical clustering

Limitations of Current Merging Schemes

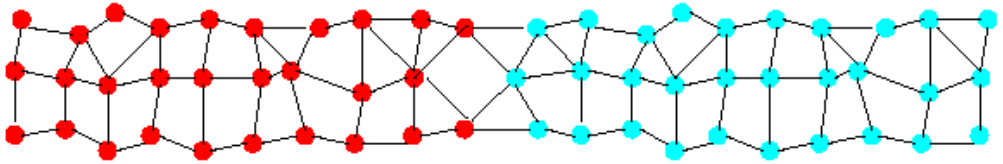
- Existing merging schemes in hierarchical clustering algorithms are **static** in nature
  - **MIN:** Merge two clusters based on their closeness (or minimum distance)
  - **Group-Average:** Merge two clusters based on their average *connectivity*

Limitations of Current Merging Schemes

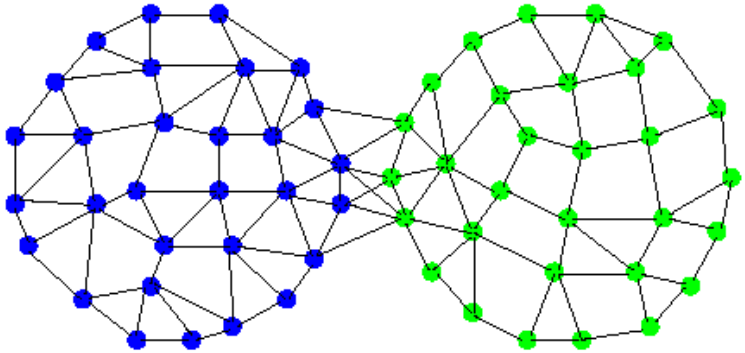




Closeness schemes will merge (a) and (b)



(a) (b)



(c) (d)

Average connectivity schemes will merge (c) and (d)

CHAMELEON: Clustering Using Dynamic Modeling

- Adapt to the characteristics of the data set to find the natural clusters
- Use a dynamic model to measure the similarity between clusters
  - Main properties are the relative closeness and relative inter-connectivity of the cluster
  - Two clusters are combined if the resulting cluster shares certain *properties* with the constituent clusters
  - The merging scheme preserves *self-similarity*

Relative Interconnectivity

- **Relative Interconnectivity (RI)** is the absolute interconnectivity of two clusters normalized by the internal connectivity of the clusters. Two clusters are combined if the points in the resulting cluster are almost as strongly connected as points in each of the original clusters. Mathematically,

$$RI = \frac{EC(C_i, C_j)}{\frac{1}{2}(EC(C_i) + EC(C_j))}, \tag{9.18}$$

where $EC(C_i, C_j)$ is the sum of the edges (of the k-nearest neighbor graph) that connect clusters C_i and C_j; $EC(C_i)$ is the minimum sum of the cut edges if we bisect cluster C_i; and $EC(C_j)$ is the minimum sum of the cut edges if we bisect cluster C_j.

Relative Closeness

- **Relative Closeness (RC)** is the absolute closeness of two clusters normalized by the internal closeness of the clusters. Two clusters are combined only if the points in the resulting cluster are almost as close to each other as in each of the original clusters. Mathematically,

$$RC = \frac{\bar{S}_{EC}(C_i, C_j)}{\frac{m_i}{m_i+m_j}\bar{S}_{EC}(C_i) + \frac{m_j}{m_i+m_j}\bar{S}_{EC}(C_j)}, \tag{9.17}$$

where m_i and m_j are the sizes of clusters C_i and C_j, respectively, $\bar{S}_{EC}(C_i, C_j)$ is the average weight of the edges (of the k-nearest neighbor graph) that connect clusters C_i and C_j; $\bar{S}_{EC}(C_i)$ is the average weight of edges if we bisect cluster C_i; and $\bar{S}_{EC}(C_j)$ is the average weight of edges if we bisect cluster C_j. (EC stands for edge cut.)

CHAMELEON: Steps

- Preprocessing Step: Represent the data by a graph
  - Given a set of points, construct the k-nearest neighbor (k-NNG) graph to capture the relationship between a point and its k nearest neighbors
  - Concept of neighborhood is captured dynamically (even if region is sparse)
- Phase 1: Use a multilevel graph partitioning algorithm on the graph to find a large number of clusters of well-connected vertices
  - Each cluster should contain mostly points from one “true” cluster, i.e., be a sub-cluster of a “real” cluster

CHAMELEON: Steps

- Phase 2: Use Hierarchical Agglomerative Clustering to merge sub-clusters
  - Two clusters are combined if the *resulting cluster shares certain properties with the constituent clusters*
- Two key properties used to model similarity:
  - Relative Interconnectivity: Absolute interconnectivity of two clusters normalized by the internal connectivity of the clusters
  - Relative Closeness: Absolute closeness of two clusters normalized by the internal closeness of the clusters

Overall Framework of CHAMELEON